

Comprehensive Assignment Brief

Object-Oriented Programming (Java) – Restaurant Ordering System

1. Introduction

This assignment requires you to design and implement a **fully functional object-oriented Java console application** for a Pizza Restaurant in Sri Lanka, similar to Pizza Hut. The objective is to apply the OOP concepts learned throughout the module (Weeks 1–6) and demonstrate your ability to translate a real-world scenario into an object-oriented software solution.

Students must work in groups and deliver a complete solution including:

- A working Java program
- UML diagrams (Use Case and Class Diagram)
- Explanation of design decisions
- Screenshots and testing evidence
- Full source code

The final system must model a small restaurant that manages menu items (pizzas, sides, drinks), processes customer orders, calculates totals, supports searching/sorting, and allows menu modifications by a manager.

Your project will be assessed based on functionality, OOP usage, UML design, code quality, documentation, and demonstration.

2. Scenario Background

You are hired by a hypothetical restaurant called “**Pizza Sri Lanka**”, inspired by Pizza Hut and similar outlets operating in Sri Lanka. The restaurant needs a lightweight internal system for managing its menu and processing customer orders through a simple computer terminal.

The system will be used by:

1. **Restaurant Staff (Manager)** – to manage menu items.
2. **Customers (via staff)** – to place pizza, sides, and drink orders.

The restaurant sells:

- **Pizzas** (vegetarian or non-vegetarian, different sizes and crust types)
- **Sides** (garlic bread, wings, etc.)
- **Drinks** (soft drinks, bottled water)

The system should maintain a list of all menu items, enable dynamic updates, and allow customers to place orders that calculate totals and apply discounts.

This assignment simulates designing the internal restaurant software using object-oriented principles.

3. Assignment Requirements (Detailed)

You must **design, implement, test, and document** a Java program that supports the features described below. Your code **MUST** use object-oriented constructs appropriately and follow the specifications strictly.

4. System Functional Requirements (Must-Have Features)

Your system **must** implement at least the following features. You may add extra features if desired, but the minimum requirements must be fully met.

4.1 Menu Management (Manager Role)

4.1.1 Add Menu Items

- The manager must be able to add new pizzas, sides, or drinks.
- When adding pizzas, manager must select:
 - Pizza Type (VEG / NON_VEG)
 - Size (SMALL / MEDIUM / LARGE)
 - Crust (THIN, REGULAR, or SECRET)
 - Price
- Items must be automatically assigned a unique ID (e.g., "I001").
- Items must be stored internally in both:
 - A **List** for display and ordering
 - A **Map** for quick access using the item ID

4.1.2 Remove Menu Items

- A manager can delete items using their unique ID.
- If an invalid ID is entered, the system must report an error.

4.2 Viewing the Menu

The system must display the complete menu in a clean, readable format using overridden `toString()` methods.

Menu should list:

- Item ID
- Name

- Type (for pizzas)
- Price
- Any additional attributes (size, crust, etc.)

4.3 Searching the Menu

The system must allow partial-name search (case-insensitive).

Examples:

- “mar” → shows Margarita Pizza
- “ch” → Chicken Wings / Chocolate Lava

4.4 Sorting Menu Items

The user must be able to sort menu items by:

- **Price (ascending)**
- **Price (descending)**
- **Name (alphabetically)**

Sorting should be implemented using Comparator or anonymous classes.

4.5 Order Management (Customer Role)

4.5.1 Create a New Order

- Customer should provide name.
- System should display menu for item selection.
- Customer can add multiple items.
- Quantities must be supported.
- Optional special instructions can be entered (e.g., “extra cheese”).

4.5.2 Order ID and Tracking

- Each order must generate a unique order ID (e.g., “ORD1001”).
- Order status must use an Enum (NEW → PLACED → PREPARING → READY → COMPLETED).

4.5.3 Payment Method

Customer must select payment type (Enum):

- CASH
- CARD

4.5.4 Order Summary

Order summary must show:

- Order ID
- Customer Name
- Order items (with quantities)
- Special notes

- Subtotal
- Optional discount
- Final Total
- Payment Method

4.6 Viewing All Orders

The system should store all completed orders and list them upon request.

5. Technical Requirements (OOP Requirements)

The following **OOP constructs** MUST be used in your project. Marks will be awarded only if they are **both implemented and explained**.

5.1 Minimum Number of Classes: 8

Recommended classes include:

- MenuItem (Parent Class)
- Pizza, Side, Drink (Child Classes)
- Order
- OrderItem
- Menu
- PizzaShopApp (Main class)

5.2 Inheritance

Must demonstrate proper inheritance where Pizza, Side, and Drink **extend** MenuItem. You must explain:

- Why inheritance is used
- What fields/methods are common
- How overriding is used

5.3 Interfaces

Include at least one interface such as Sellable.

Explain:

- Why an interface is needed
- Which classes implement it
- What behaviors are abstracted

5.4 Method Overloading

You must write **two or more versions** of the same method with different parameters.

Example:

```
addItem(MenuItem item);
addItem(MenuItem item, int quantity);
```

Explain why and where overloading is used.

5.5 Method Overriding

Every subclass should override `toString()`.

Explain how overriding supports polymorphism and improves readability.

5.6 Enum Types

You must create **three or more enum types**, e.g.:

- `PizzaSize`
- `Crust`
- `PizzaType`
- `PaymentMethod`
- `OrderStatus`

Explain why enumerations are better than string constants.

5.7 Collections: List & Map

You must use:

- **`ArrayList`** for storing lists of items or orders
- **`HashMap`** for fast lookup using keys

Explain:

- Difference between List and Map
- Why each was selected

5.8 Polymorphism

Your system must demonstrate:

- Storing different MenuItem subclasses in the same List
- Calling overridden methods at runtime (dynamic binding)

Explain how polymorphism helps reduce code duplication.

6. UML Diagrams (Mandatory)

6.1 Use Case Diagram

Must include:

Actors:

- Customer
- Manager

Use Cases:

- Add Menu Item
- Remove Menu Item
- View Menu
- Search Menu
- Sort Menu

- Create Order
- Add Items to Order
- Choose Payment Method
- Confirm Order
- View All Orders

Use correct UML notation:

- Actor icons
- Use case ovals
- Include, extend relationships (if any)

6.2 Class Diagram

Must include:

- All classes
- All attributes (with visibility symbols + - #)
- Key methods
- Inheritance (arrows)
- Interfaces (dashed arrows)
- Associations (e.g., Order *has many* OrderItems)
- Multiplicities (e.g., 1..*, 0..1)

7. Testing Requirements

You must include testing evidence:

7.1 Screenshots Required

- Program start screen
- Menu listing
- Adding menu items
- Removing items
- Searching
- Sorting
- Order creation steps
- Final order summary
- Order listing

7.2 Testing Notes

You must write a **brief paragraph** explaining:

- What you tested
- Expected vs observed output
- Any error-handling

8. Documentation Requirements (Submission Report)

Your **final report** must contain:

1. **Cover Page**
 - Student name(s), ID(s), Batch, Course
2. **Introduction**
 - Overview of the system and its purpose
3. **Scenario Explanation**
 - Description of how the restaurant operates
 - How your system supports these operations
4. **UML Use Case Diagram**
5. **UML Class Diagram**
6. **Use Case Descriptions** (recommended)
 - Brief steps for each major use case
7. **OOP Explanation Section**
 - Detailed explanation of how each OOP concept is applied
8. **Screenshots of Your System Output**
9. **Summary and Conclusion**
 - Lessons learned
 - Challenges faced
10. **Appendix**
 - Source code OR link to GitHub repository

9. Source Code Submission Rules

You must submit:

- All .java source files
- No .class files
- Program **must compile without errors**
- Code must be properly indented and commented
- No external libraries allowed (except Java SDK)
- Use only features taught in Weeks 1–6

10. Viva / Demonstration Requirements

You must demonstrate:

- How your system runs
- Explain your code structure
- Explain the diagrams
- Justify design decisions

Each member must be able to answer questions individually.
Failure to demonstrate understanding may result in mark reduction.

11. Marking Scheme

Component	Description	Marks
Analysis	Clear scenario understanding	5
Use Case Diagram	Correct actors, use cases, connections	5
Class Diagram	Proper structure, associations, inheritance	5
OOP Concepts	Inheritance, interface, enums, overriding, overloading	10
Collections Usage	Correct use of List and Map	5
Functionality	Program runs fully as required	10
Testing & Screenshots	Evidence and explanation	10
Code Quality	Style, comments, readability	5
Documentation	Complete, professional report	5
Viva	Presentation and answering of questions	20
Code modification		20
Total		100

You need to upload the full code as well as the report that is created and for the viva on the 5th of May 2026 each group needs to bring the printed report with the working code and each member will have to explain the code. Questions will be asked from any part of the code, use case diagram and class diagram. A code modification will be given and need to be completed within the given time frame.